GRAPH-OF-THOUGHTS FOR MULTI-STAGE ATTACK INVESTIGATION: COMPARING CHAIN, TREE, AND GRAPH REASONING FOR KILL-CHAIN RECONSTRUCTION

A Project Thesis

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Artificial Intelligence Engineering

Submitted by

Hari Vaarthan T D (CB.SC.U4AIE23228)
Joseph Binu (CB.SC.U4AIE23235)
Jerome Richard D (CB.SC.U4AIE23265)
Sajeev Senthil (CB.SC.U4AIE23272)

Team C02

Under the guidance of

Dr. Prajisha C

Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham, Coimbatore – 641112

October 2026

BONAFIDE CERTIFICATE

This is to certify that the project thesis entitled **"Graph-of-Thoughts for Multi-Stage Attack Investigation: Comparing Chain, Tree, and Graph Reasoning for Kill-Chain Reconstruction"** is a bonafide record of the work carried out by **Hari Vaarthan T D (CB.SC.U4AIE23228), Joseph Binu (CB.SC.U4AIE23235), Jerome Richard D (CB.SC.U4AIE23265) and Sajeev Senthil (CB.SC.U4AIE23272)** under my guidance and supervision, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence Engineering at the Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore.

Dr. Prajisha C	**Chairperson**
Project Guide	Amrita School of Artificial Intelligence

Submitted for the project review held on _____

Internal Examiner	External Examiner

DECLARATION

We hereby declare that the project thesis entitled "Graph-of-Thoughts for Multi-Stage Attack Investigation: Comparing Chain, Tree, and Graph Reasoning for Kill-Chain Reconstruction" is the result of our own work, carried out under the guidance of Dr. Prajisha C, Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore. To the best of our knowledge, this work has not been submitted, in part or in full, for the award of any other degree or diploma. Wherever we have used the work of others, we have acknowledged it in the text and listed it in the references.

Place: Coimbatore
Date: _____

Name	**Signature**
Hari Vaarthan T D (CB.SC.U4AIE23228)	
Joseph Binu (CB.SC.U4AIE23235)	
Jerome Richard D (CB.SC.U4AIE23265)	
Sajeev Senthil (CB.SC.U4AIE23272)	

ACKNOWLEDGEMENT

We thank our guide, Dr. Prajisha C, for steering this project from a broad idea about reasoning structures to a question we could actually test, and for pressing us to report results that did not support our hypothesis rather than quietly reshaping the study around them.

We are grateful to the Chairperson and the faculty of the Amrita School of Artificial Intelligence for the courses that supplied the foundations used throughout, and to the department for access to the GPU cluster on which every experiment was run.

We thank the authors of the NODLINK Simulated-Data release, whose attack logs and annotations made a controlled comparison possible, and the authors of the Graph of Thoughts framework, whose open implementation we built on.

Finally, we thank our families and friends for their patience during the long evenings spent waiting on batch jobs.

Hari Vaarthan T D
Joseph Binu
Jerome Richard D
Sajeev Senthil

ABSTRACT

Reconstructing a multi-stage cyber attack from host audit logs is still largely manual work. Structured prompting topologies (Chain-, Tree- and Graph-of-Thoughts) improve language-model reasoning on short, well-defined tasks, but it is unknown whether the extra structure helps when the evidence is a long and noisy provenance log. This thesis isolates that question. We build a pipeline in which ground truth, evidence construction, reduction, serialization, prompt, parser and scorer are identical for the three topologies, so the topology is the only variable. It is evaluated on five attack scenarios from the NODLINK Simulated-Data release (89 annotated attack events) with locally hosted open models served by vLLM and no training or fine-tuning. Our hypothesis was that the Aggregate operation of Graph-of-Thoughts would raise stage-level recall.

On the metric named in the hypothesis, stage recall, GoT was not reliably better than CoT or ToT: 2 of 15 paired comparisons fell below $p = 0.05$ before correction and none survived it. GoT did recover more events (uncorrected $p < 0.05$ in 5 of 10 comparisons) at lower precision for the smaller models. Reading raw outputs against ground truth explained why: only 15.3% of true attack events were ever detected, and exact stage labels were right about 20% of the time, because each model carries a stable stage-label prior that does not match the data. Adding MITRE ATT&CK stage definitions to the shared prompt doubled exact stage recall (0.056 to 0.123, Wilcoxon $p = 0.0015$); per-process triage reduced false positives (precision 0.245 to 0.329, $p = 0.04$); and scaling Qwen 2.5 from 7B to 32B raised exact stage recall from 0.15 to 0.30 ($p = 0.012$). Graph-of-Thoughts had the highest event F_1 and stage recall for both large models. A security-tuned 8B model scored below the general model (event F_1 0.101 against 0.295), partly because 38% of its runs gave no parseable answer, and did not separate the topologies. On the longest inputs a 70B model scored below an 8B model, which shows input length is a confound in topology comparisons. We conclude that at the 7–70B scale, detection recall and stage calibration matter more than reasoning topology, and we propose a segmented-Chain-of-Thought control to separate the benefit of splitting from the benefit of aggregating.

Contents

Bonafide Certificate . . . . . **i**

Declaration . . . . . **ii**

Acknowledgement . . . . . **iii**

Abstract . . . . . **iv**

List of Figures . . . . . **viii**

List of Tables . . . . . **ix**

List of Abbreviations and Symbols . . . . . **x**

1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Problem Statement 2

1.4 Research Questions and Hypothesis 2

1.5 Objectives 2

1.6 Scope 3

1.7 Contributions 3

1.8 Organisation of the Thesis 3

2 Literature Review **4**

2.1 Theme A: Reasoning Topologies 4

2.2 Theme B: Attack Reconstruction on Provenance Logs 5

2.3 Theme C: Language-Model Agents for Security Investigation 5

2.4 Research Gap 5

2.5 Summary 6

3 Cyber-Security Background **7**

3.1 Advanced Persistent Threats and the Kill Chain 7

3.2 A Worked Example: the WS12 Attack 7

3.3 Host Audit Logs and Provenance . . . 8
3.4 Indicators and Common Attacker Tooling . . . 8
3.5 Detection versus Investigation . . . 8
3.6 Why Evaluation Needs Annotated Attacks . . . 8

4 Reasoning Topologies, Hypothesis and Tests . . . 10
4.1 The Three Topologies . . . 10
4.2 Why Aggregation Should Help: the Formal Argument . . . 12
4.3 What the Hypothesis Predicts . . . 12
4.4 Tests Performed . . . 12

5 Proposed System: Pipeline Design . . . 14
5.1 Ground-Truth Parser . . . 14
5.2 Evidence Graph and Reduction . . . 15
5.3 Serializer . . . 15
5.4 Shared Prompt and Parser . . . 15
5.5 Method Implementations . . . 17
5.6 Improvement Variants . . . 17
5.7 Scorer . . . 17

6 Data, Evaluation Protocol and Implementation . . . 18
6.1 Dataset . . . 18
6.2 Models . . . 21
6.3 Experimental Grid . . . 21
6.4 Evaluation Metrics . . . 21
6.5 Statistics . . . 23
6.6 Software and Hardware . . . 23

7 Verification, Audit and Corrections . . . 24
7.1 Defects in Data Handling . . . 24
7.2 Defects in the Scorer . . . 24
7.3 Defects in the Methods . . . 25
7.4 Defects in Infrastructure . . . 25
7.5 Claims Withdrawn or Qualified . . . 25

8 Results . . . 26
8.1 Experiments Run . . . 26
8.2 Baseline: Topology Comparison (RQ1) . . . 26
8.3 Tests of the Hypothesis . . . 28
8.4 Comparison across Models and Topologies . . . 29

8.5 Improvement Rounds . . . 30
8.6 Scaling the Model . . . 32
8.7 Security-Tuned Model (RQ3) . . . 32
8.8 Accuracy and Cost . . . 34
8.9 Summary of Results . . . 34

9 Discussion: Where and Why Reconstruction Fails . . . 35
9.1 Answers to the Research Questions . . . 35
9.2 Failure 1: Most Attack Events Are Never Detected . . . 35
9.3 Failure 2: Stage Labels Are Miscalibrated . . . 35
9.4 Failure 3: Evidence the Method Cannot Use or Cannot Tell Apart . . . 37
9.5 Failure 4: Input Length and Output Format . . . 38
9.6 Failure 5: Unannotated Activity Counts as an Error . . . 38
9.7 Status of the Claims . . . 39
9.8 Threats to Validity . . . 39
9.9 What the Audit Taught Us . . . 40

10 Conclusion and Future Work . . . 41
10.1 Conclusion . . . 41
10.2 Objectives Revisited . . . 41
10.3 Future Work . . . 42
10.4 Practical Takeaways . . . 42

References . . . 43

A Configuration . . . 44

B Reproducing the Results . . . 45
B.1 Environment . . . 45
B.2 Pipeline . . . 45
B.3 Grids . . . 45
B.4 Figures . . . 45

C Additional Outputs . . . 46
C.1 Per-Scenario Baseline . . . 46

List of Figures

4.1 The three reasoning topologies. E is the evidence, A the answer, Agg the Aggregate step and Ref the Refine step. 11

5.1 Kill-chain reconstruction pipeline. The figure is the design schematic: the token range and the number of models shown in it are planning values. The realised values are 16K to 58K tokens per scenario (Table 6.3) and two base models in the main grid (Table 6.5). 14
5.2 Evidence graph of the WS12 scenario after reduction (filtered view). Process nodes, external IP nodes and file-write nodes are drawn in different colours; parent-to-child process edges follow the process tree. 16

8.1 Baseline mean event F_1, stage recall and event recall by method. 27
8.2 Baseline event recall and exact stage recall by scenario and method. 28
8.3 Event F_1, event recall and exact stage recall for each model and topology under the v4 setup. 29
8.4 Pooled precision, event recall and stage recall by variant (63 runs each). . . 31
8.5 Llama 70B against Llama 8B without triage, split by input length. At 58K tokens the larger model's recall falls below the smaller one's. 32
8.6 Share of Llama-Primus-Reasoning runs that produced a parseable answer, by method (15 runs each). 33
8.7 Stage recall against cost for each method and model, from the baseline grid. 34

9.1 Stage confusion matrix for Llama 3.1 8B: true stage against predicted stage over all found events. 36
9.2 Stage confusion matrix for Qwen 2.5 7B. 37

List of Tables

2.1 Work on language-model reasoning topologies. 4
2.2 Work on automated attack reconstruction. 5

3.1 Stage names in the ground truth and their meaning. 7

4.1 Average cost per run in the baseline grid (21 runs per method). 11
4.2 Latency and volume of the final thought for each topology [5]. 12

6.1 Scenarios, annotated attack events and the models run in the baseline grid. . 18
6.2 Raw log size and the effect of reduction. "Process starts" are process-creation events; "collapsed" counts exact repeats merged into one node. . . 19
6.3 Composition of the reduced evidence graph and the length of its serialization. 19
6.4 Ground-truth events per stage and scenario. 20
6.5 Models and the constraints that decided their use. 21
6.6 Software and hardware. 23

8.1 Inventory of experiments. 26
8.2 Baseline results by method (mean of 21 runs each). 26
8.3 Baseline results by model and method. 27
8.4 Paired Wilcoxon tests between topologies (generated). Δ is the mean difference; n is the number of matched (scenario, model, seed) cells. 28
8.5 Models and topologies under the v4 setup (generated). 30
8.6 Paired tests between successive variants (generated). 31
8.7 Llama-Primus-Reasoning by method, v4 setup (generated). 32
8.8 Llama-Primus-Reasoning against Llama 3.1 8B, v4 setup, 45 matched runs each. 33

9.1 What the evidence supports. 39

10.1 Status of the objectives of Chapter 1. 41

A.1 Per-model settings used in the grids. 44

C.1 Baseline means by scenario (all methods and models pooled). 46

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviation	**Meaning**
APT	Advanced persistent threat
ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
AWQ	Activation-aware weight quantization
C2	Command and control
CoT, ToT, GoT	Chain-, Tree- and Graph-of-Thoughts
ETW	Event Tracing for Windows
IOC	Indicator of compromise
LLM	Large language model
NODLINK	Provenance-based APT detection system and its Simulated-Data benchmark [1]
SOC	Security operations center
vLLM	High-throughput LLM inference engine
Symbol	**Meaning**
x	Serialized evidence given to a reasoning method
y	Answer of a method: reported events with stages, and a narrative
P, R, F_1	Event-level precision, recall and F_1
SR	Exact stage recall
H	Hallucination rate
p	p-value of a paired Wilcoxon signed-rank test

CHAPTER 1

INTRODUCTION

1.1 Background

Modern enterprise hosts generate on the order of 10^5 to 10^6 log events per day: process starts, file operations and network connections. A multi-stage attack (initial access, execution, discovery, credential theft, lateral movement and persistence) leaves only a handful of malicious events inside that volume. In the five attack scenarios used in this thesis the attack is between 6 and 37 annotated events, hidden in logs of between 1.1 and 3.8 million raw events. Turning those scattered events into a coherent, stage-labelled account of the intrusion is called *kill-chain reconstruction*. Security operations centers (SOCs) still do much of it by hand, and it is among the most time-consuming tasks an analyst performs.

1.2 Motivation

Two developments make the question timely.

Industry driver. Cloud intrusion volume grew by roughly 75% between 2022 and 2023 [2], while analyst headcount has not grown in proportion. Automated investigation support is therefore a near-term operational need.

Technology driver. Open-weight language models such as Llama 3.1 and Qwen 2.5 are now capable enough to run on local hardware and read structured evidence, which matters when logs are sensitive. A family of *prompting topologies* has also appeared: Chain-of-Thought (CoT) [3], Tree-of-Thoughts (ToT) [4] and Graph-of-Thoughts (GoT) [5]. These do not change what the model knows; they change how it may combine what it is given. GoT adds an *Aggregate* operation that merges several independently reasoned partial results into one. Attack evidence is scattered: a file written early may be executed much later, and the same external address may appear in unrelated parts of a log. That makes attack investigation a natural test for aggregation. Whether the structure helps on a real investigative task, and not only on sorting and puzzle benchmarks, has not been tested under controlled conditions.

1.3 Problem Statement

Systems for automated attack reconstruction, such as OCR-APT [6], rely on a separately trained detector to isolate malicious subgraphs and use the language model only to narrate what the detector found. It is therefore unknown whether a language model's own reasoning can perform the investigative step unassisted, and whether the topology of that reasoning changes reconstruction quality. This work removes the trained detector, holds the evidence, its serialization and the model weights fixed, and measures kill-chain reconstruction quality as a function of reasoning topology.

1.4 Research Questions and Hypothesis

RQ1 With evidence, prompt, model and scoring held fixed, does GoT achieve higher stage-level recall than CoT and ToT?

RQ2 If not, where does reconstruction fail, and which factors (prompt vocabulary, evidence reduction, model size, input length) change the results?

RQ3 Does a security-tuned model of the same size and architecture change the conclusion?

Hypothesis H1. Graph-of-Thoughts, through its Aggregate operation, reconstructs multi-stage attacks with higher stage-level recall than Chain-of-Thought or Tree-of-Thoughts. Aggregation can connect attack steps that look benign in isolation but become clearly malicious once linked, which a chain or a tree cannot do in one step.

Null hypothesis H0. Stage-level recall does not differ significantly among CoT, ToT and GoT once evidence, model weights and prompt content are fixed. Any observed difference is noise.

1.5 Objectives

1. Build a fixed evidence pipeline that reduces each host's log of about one million events to a candidate set serialized identically for all three methods, verified by a hard gate that 100% of ground-truth attack events survive reduction.
2. Implement CoT, ToT and GoT on the same local models with the `graph-of-thoughts` framework, and obtain valid, parseable structured output for every scenario, method and model combination.
3. Score reconstruction with event-level precision, recall and F_1, a stage-level recall, a hallucination rate and cost measures, as means over several seeds.
4. Test H1 with paired significance tests across scenarios, seeds and models.

5. Characterise the accuracy–cost trade-off of each topology.
6. Diagnose the failures and test targeted improvements one at a time.

1.6 Scope

In scope: inference-only comparison of CoT, ToT and GoT on a fixed, rule-based candidate evidence set; the NODLINK Simulated-Data benchmark (five multi-stage attacks on Windows Server 2012, Ubuntu and Windows 10); open-weight local models with no fine-tuning; event-level, stage-level and cost evaluation.

Out of scope: training or fine-tuning any model; building a new detector (the reduction step is deliberately structural and identical for all methods); real-time detection; claims beyond the five scenarios evaluated.

1.7 Contributions

- A controlled experimental design in which all three topologies receive identical evidence and are scored identically (Chapters 5 and 6).
- A result that GoT's stage recall is not reliably higher than CoT's or ToT's at 7B to 70B model scale, while stage definitions in the prompt, evidence triage and model size have clearer effects (Chapter 8).
- A failure analysis based on reading raw outputs against ground truth, identifying detection recall, stage calibration, input length and output-format failures as the limiting factors.
- A record of the defects found and fixed during the work, several of which would have invalidated the comparison silently (Chapter 7).

1.8 Organisation of the Thesis

Chapter 2 reviews related work and states the research gap. Chapter 3 gives the cybersecurity background. Chapter 4 defines the three reasoning topologies, the hypothesis and the statistical tests. Chapter 5 describes the pipeline. Chapter 6 specifies the data, models, metrics and protocol. Chapter 7 records verification and corrections. Chapter 8 reports the results, Chapter 9 explains where and why the methods fail, and Chapter 10 concludes.

CHAPTER 2

LITERATURE REVIEW

The literature is grouped into three themes: reasoning topologies, which are the independent variable of this study; automated attack reconstruction on provenance and audit logs, which is the task; and language-model agents for security investigation, which is the closest contrast case.

2.1 Theme A: Reasoning Topologies

Table 2.1: Work on language-model reasoning topologies.

Work	Method	Reported result	Limitation for this task
Wei et al. [3], NeurIPS 2022	Single generation with intermediate reasoning steps before the answer	Large gains over direct prompting on arithmetic, commonsense and symbolic reasoning	No mechanism to revisit an early wrong step; errors propagate to the answer
Yao et al. [4], NeurIPS 2023	Generate, self-evaluate and prune over branching thoughts	Game of 24 success from 4% (CoT) to 74% (ToT) with GPT-4	Each node has one parent, so evidence explored in a pruned branch is lost to the others
Besta et al. [5], AAAI 2024	Adds Aggregate (many parents, one child) and Refine (self-loop) to Generate and Evaluate	+70% quality over CoT and +62% over ToT on sorting, with over 31% lower cost than ToT	Evaluated only on abstract tasks with clean, fully specified inputs, never on noisy real evidence
Zhang et al. [7], NeurIPS 2024	Fine-tunes a model on ToT's search preferences so a single chain approximates ToT	Chain-level inference cost with tree-level accuracy after fine-tuning	Needs training, which an inference-only design excludes; does not test aggregation

2.2 Theme B: Attack Reconstruction on Provenance Logs

Table 2.2: Work on automated attack reconstruction.

Work	Method	Reported result	Limitation for this task
Li et al. [1], NDSS 2024	Online provenance-graph anomaly detection and investigation; releases the Simulated-Data benchmark used here	High detection accuracy at production-level throughput; event-level F_1 of about 0.248 reported in later comparative work	No language-model reasoning; output is a graph, not a stage-labelled narrative
Aly et al. [6], CCS 2025	A trained graph network flags anomalous subgraphs; a language model then narrates the flagged subgraph	F_1 of about 0.96 on this dataset family, above NODLINK	The model never detects or links; it summarises what a separate detector isolated, so reasoning topology is not a variable

2.3 Theme C: Language-Model Agents for Security Investigation

Wu et al. [8] evaluate agents that issue SQL queries against 57 Microsoft Sentinel log tables, graded against investigation-graph ground truth (7,542 generated questions, 589 in the test split). The best model reaches a reward of 0.606. The authors report that query-writing success drives most of the reward variance, so the benchmark cannot separate investigative reasoning from tool-use skill. This is the contrast case for our design, which gives every method the same evidence and no tools.

2.4 Research Gap

1. **No work isolates reasoning topology as the only variable in attack reconstruction.** OCR-APT removes the question by giving detection to a trained network. ExCyTIn-Bench lets the model reason but confounds reasoning with query skill. No benchmark holds evidence, model and detection fixed while varying only chain, tree or graph structure on a multi-stage attack task.
2. **GoT's aggregation has not been tested where it should matter most.** The topologies were validated on sorting, keyword counting and Game of 24. None of these has the structure of attack investigation: causally related evidence scattered across a large noisy log, where each piece looks weak alone and becomes conclusive only when merged with a separately reasoned piece.
3. **The chain-to-tree trade-off has been attacked by fine-tuning, not by topology**

comparison under a no-training constraint. Whether GoT's aggregation adds something a tree cannot express, with weights fixed, is the more diagnostic question about why structure matters.

2.5 Summary

Prior work establishes the topologies, the benchmark and detector-assisted reconstruction separately. This thesis combines them into one controlled comparison: an inference-only test of chain, tree and graph reasoning on real provenance evidence, with significance testing across models and scenarios.

CHAPTER 3

CYBER-SECURITY BACKGROUND

This chapter introduces the security concepts the rest of the thesis relies on, for readers whose background is machine learning.

3.1 Advanced Persistent Threats and the Kill Chain

An advanced persistent threat (APT) is a well-resourced attacker who stays inside a network for an extended period to reach a specific goal. APT operations are staged: each step prepares the next and may be separated from it by minutes or days. A *kill chain* describes the attack as an ordered sequence of such stages. The MITRE ATT&CK knowledge base [9] catalogues the attacker's goals at each stage as *tactics* and the concrete methods as *techniques*. The ground truth of our dataset names ten tactics, listed in Table 3.1 with the short definitions used in our stage-definition prompt variant.

Table 3.1: Stage names in the ground truth and their meaning.

Stage (tactic)	**Attacker's goal**
Initial Access	Get a first foothold on the host, for example by delivering and running a backdoor
Execution	Run attacker-controlled code or commands
Persistence	Keep access across reboots and logouts, for example through registry keys or scheduled tasks
Privilege Escalation	Obtain higher rights than the initial foothold
Defense Evasion	Avoid detection, for example by disabling or bypassing security tools
Credential Access	Steal account names and passwords or their hashes
Discovery	Learn about the system and network, for example by scanning for other hosts
Lateral Movement	Move to other machines using stolen credentials or remote execution tools
Collection	Gather data of interest before exfiltration
Command and Control	Communicate with attacker-run infrastructure to receive instructions

3.2 A Worked Example: the WS12 Attack

The smallest scenario, WS12, shows how a kill chain appears in a log. We analysed it by hand before any automation. Its six annotated events map to stages as follows: recon-

naissance and download of a backdoor (Initial Access); registry, firewall and sticky-keys changes (Persistence); a brute-force tool (Credential Access); `nbtscan` (Discovery); and `PAExec` against a second host (Lateral Movement). Individually several of these are ordinary administrative commands. Only the sequence, and in some cases a shared file or network address, shows that they belong together. This is the property the Aggregate operation is meant to exploit.

3.3 Host Audit Logs and Provenance

Host monitoring records system activity as events. On Windows, Event Tracing for Windows (ETW) provides process-start, TCP/IP and file-write events; on Linux, tools such as sysdig and Falco record system calls. A *provenance graph* turns these events into a graph whose nodes are system entities (processes, files, network endpoints) and whose edges are causal relations (a process spawns a process, writes a file, connects to an address). Provenance graphs support forensic questions such as how a given file came to exist. Their weakness is volume: the raw logs used here hold between 1.1 and 3.8 million events per host.

3.4 Indicators and Common Attacker Tooling

An indicator of compromise (IOC) is an observable that suggests intrusion, such as an external IP address, a file name or a command line. Attackers often use legitimate system utilities (for example `certutil` to download files) because these blend into normal activity. Recognising that a given utility is being used maliciously depends on knowledge of attacker practice and on context, which is why the task is difficult for a general language model and why we test a security-tuned model in RQ3.

3.5 Detection versus Investigation

Detection answers whether and where something suspicious occurred; investigation answers what happened, in which order and through which stages. Trained provenance detectors address the first [1]. This thesis addresses the second without a trained detector, by asking a language model to read a structurally reduced evidence set and name the attack events with their stages.

3.6 Why Evaluation Needs Annotated Attacks

Scoring an investigation requires knowing which events are truly part of the attack. The NODLINK Simulated-Data benchmark provides this because the attacks were executed by

the dataset's authors from known plans, and each step is annotated with its command and stage. That is why we use it in preference to flow-level intrusion-detection datasets, which carry one label per network flow and no command-line kill chain to reconstruct.

CHAPTER 4

REASONING TOPOLOGIES, HYPOTHESIS AND TESTS

This chapter defines the three reasoning topologies as they are implemented, states what the hypothesis predicts, and describes the statistical tests used to confirm or reject it. The notation is deliberately small: x is the serialized evidence, y is a method's answer (the reported attack events with stages, plus a narrative), and each model call is written as $\mathrm{LLM}(\cdot)$.

4.1 The Three Topologies

4.1.1 Chain-of-Thought

CoT makes one model call over the whole evidence:

$$y = \mathrm{LLM}(x). \tag{4.1}$$

The model reasons step by step inside that single generation. It cannot revisit an earlier step, and every part of x competes for attention in one long context.

4.1.2 Tree-of-Thoughts

ToT samples $k = 3$ candidate answers from the same prompt, asks the model to score each candidate against the evidence on a scale from 0 to 1, and keeps the best:

$$y_i = \mathrm{LLM}(x), \quad s_i = \mathrm{Score}(x, y_i) \in [0,1], \quad y = y_{i^*}, \;\; i^* = \arg\max_i s_i. \tag{4.2}$$

One generation call yields the three candidates and three scoring calls rate them, so a ToT run makes four calls. Candidates do not see each other, and everything found by a discarded candidate is lost.

4.1.3 Graph-of-Thoughts

GoT [5] models reasoning as a graph in which a node may have several parents. Our graph has four steps:

$$\text{Split:} \quad x \to x_1, \ldots, x_4 \quad \text{(four segments of similar text size)}, \tag{4.3}$$

$$\text{Generate:} \quad z_i = \mathrm{LLM}(x_i), \quad i = 1, \ldots, 4, \tag{4.4}$$

$$\text{Aggregate:} \quad a = \mathrm{LLM}(z_1, z_2, z_3, z_4) \quad \text{(many parents, one child)}, \tag{4.5}$$

$$\text{Refine:} \quad y = \mathrm{LLM}(x, a). \tag{4.6}$$

Each segment is analysed without seeing the others, so the Aggregate call is the only place where findings from different parts of the log meet. Refine sees the full evidence together with the merged answer. In the v5 variant the Aggregate call also receives a short list of structural links between segments: an external address contacted from more than one segment, and a file written in one segment whose name appears in a process launched in another. A GoT run makes six calls (four, one, one), fewer when a log yields fewer segments. If the Refine output cannot be parsed, the merged answer is kept.

Figure 4.1 contrasts the three structures and Table 4.1 gives their measured cost.

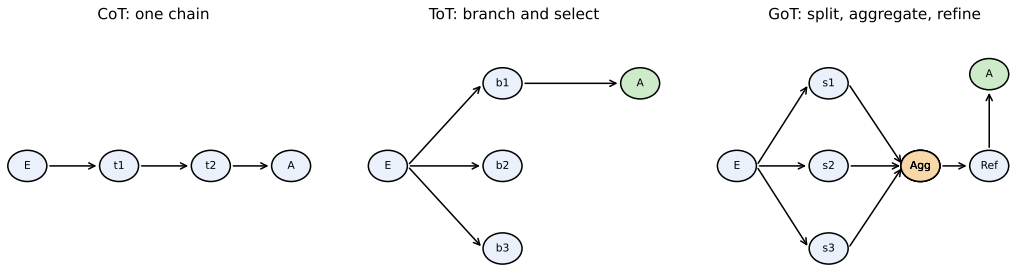


Figure 4.1: The three reasoning topologies. E is the evidence, A the answer, Agg the Aggregate step and Ref the Refine step.

Table 4.1: Average cost per run in the baseline grid (21 runs per method).

Method	LLM calls	Tokens	Wall clock (s)
CoT	1.0	37,467	25.8
ToT	4.0	149,666	40.4
GoT	5.4	81,461	103.4

GoT uses fewer tokens than ToT because most of its calls read only a quarter of the evidence, but it takes the longest wall-clock time because its steps run one after another.

4.2 Why Aggregation Should Help: the Formal Argument

Besta et al. [5] define the *volume* of a thought as the number of earlier thoughts from which it is reachable, that is, how much prior reasoning could have informed it. For N thoughts and branching factor k:

Table 4.2: Latency and volume of the final thought for each topology [5].

Topology	Latency (longest path)	Volume of final thought
CoT	N	N
ToT	$\log_k N$	$O(\log_k N)$
GoT	$\log_k N$	N

GoT is the only scheme with both low latency, like a tree, and full volume, like a chain, because its aggregation node is reachable from every independently reasoned segment, whereas a tree keeps only the one surviving path. This is why H1 predicts an advantage specific to aggregation and not merely from branching.

4.3 What the Hypothesis Predicts

1. **Main effect.** Stage recall of GoT exceeds that of ToT, which exceeds that of CoT, aggregated over scenarios and seeds, with the GoT–ToT gap statistically distinguishable. This isolates aggregation, because ToT already has branching and evaluation.
2. **Ablation control.** GoT with Aggregate outperforms GoT without Aggregate (independent segment answers, best one kept) at equal or larger call budget. This would separate aggregation from simply making more calls.
3. **Cost.** GoT is expected to cost the most. Cost is reported separately and is not part of the accuracy claim.

H1 is rejected if GoT's stage recall is statistically indistinguishable from, or below, that of ToT and CoT.

4.4 Tests Performed

Paired topology comparison (prediction 1). For a given experiment, each method is run on the same scenarios, models and seeds, so a run of one method can be paired with a run of another. For each pair of methods we form the per-cell differences $d_j = m_j(\mathrm{A}) - m_j(\mathrm{B})$ of a metric m (stage recall is the primary metric, event recall secondary) and apply the two-sided Wilcoxon signed-rank test [10] to d_j. The null hypothesis is that the median difference is zero. We report the mean difference Δ, the p-value and the number of pairs

n. A comparison is called significant only at $p < 0.05$. The test is non-parametric because scores are bounded and many are zero, and paired because scenario difficulty varies far more than method differences. Each seed of a scenario counts as one pair, which treats repeated runs as independent and makes the test somewhat anti-conservative. Because we test three comparisons on two metrics in five experiments (30 tests), we also state which results survive Bonferroni and Holm correction.

Paired variant comparison. The same test compares an earlier and a later variant of the pipeline (for example before and after adding stage definitions) over matched (scenario, model, method, seed) cells.

Diagnostic checks. Because a non-significant result cannot distinguish "no effect" from "no power", we also read raw outputs against ground truth. This produced a stage confusion matrix, a stage-label distribution per model, a split of results by input length, and a parse-success analysis (Chapter 9).

Not performed. The ablation control of prediction 2 (GoT without Aggregate) was not run. The nearest experiment is v5, which changes what Aggregate receives, not whether it exists. A related control, segmented CoT, is proposed as future work (Chapter 10).

CHAPTER 5

PROPOSED SYSTEM: PIPELINE DESIGN

Figure 5.1 shows the experimental pipeline. Every stage is fixed and identical across conditions except the reasoning layer, so a measured difference in output quality is attributable to the reasoning topology alone. The figure also separates two graphs that must not be confused: the *evidence graph* (steps 01–03), built once and shared by all methods, and the *thought graph* built inside the reasoning layer.

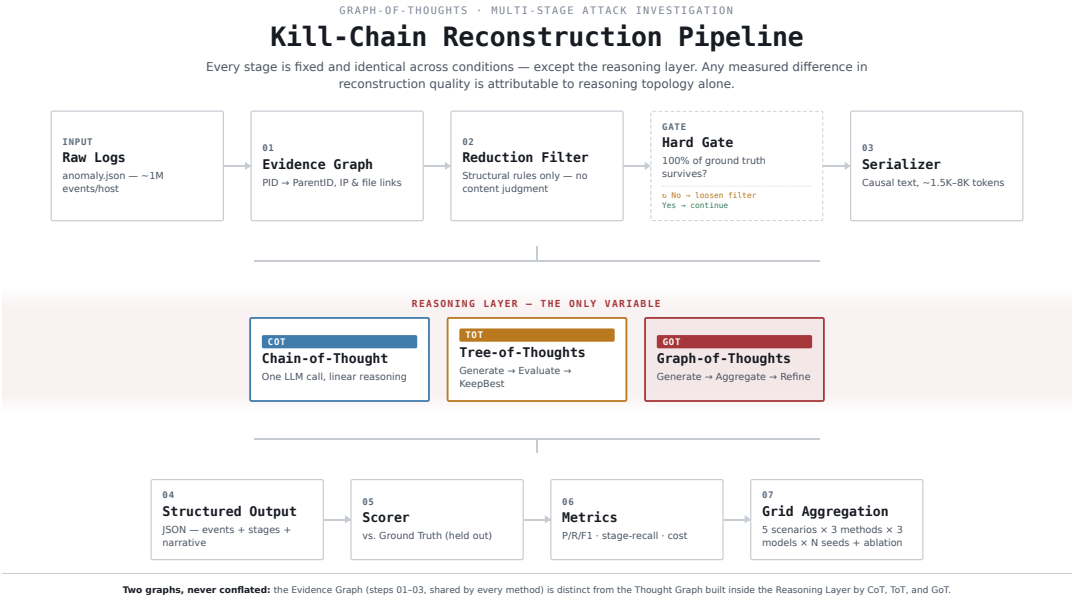


Figure 5.1: Kill-chain reconstruction pipeline. The figure is the design schematic: the token range and the number of models shown in it are planning values. The realised values are 16K to 58K tokens per scenario (Table 6.3) and two base models in the main grid (Table 6.5).

5.1 Ground-Truth Parser

For each scenario, the attack annotation files are parsed into a list of nodes. Each node has a stage and a match substring, for example a fragment of the attack command. The pairing of stage and substring was resolved by hand, scenario by scenario, because the identifier join between the annotation files and the attack-analysis spreadsheet differs between scenarios (Section 6.1.5). All 89 nodes have a verified pair. Ground truth is read only by the scorer and is never shown to a model.

5.2 Evidence Graph and Reduction

The evidence graph has three node types: processes, external IP addresses and files. Edges are of three kinds: `spawned` (parent process to child), `connected_to` (process to an external address) and `wrote` (process to a suspicious file path). Windows scenarios are parsed from ETW-style records and the Ubuntu scenario from sysdig/Falco-style system-call records, which have a different schema, so there are two parsers. Process nodes are keyed on process identity including start time rather than on the PID alone, because PIDs are reused.

Reduction keeps all process-start nodes, external network connections and suspicious file writes, and collapses exact repeats of the same image, command line and parent. It uses only event type and external-address rules. It does not judge whether a process looks malicious. If it did, the pipeline would contain a hidden detector and the claim that the language model performs the investigation would be false. The dataset's own label field `is_warn` is never read.

Figure 5.2 shows the evidence graph built for the WS12 scenario. Only the branches that touch an external address or a suspicious write, with the process ancestry needed to reach them, are drawn, because the full graph of 234 nodes is not legible as a static image. The complete graph is stored in the repository.

Hard gate. After reduction we check that every ground-truth match substring occurs in the command lines of the reduced set. All five scenarios pass. Documented exceptions are the two nodes that cannot be checked because the ground truth carries no match text, and one Ubuntu node that comes from a second host reached by lateral movement. A scenario that failed the gate would not be run, because recall would then be capped by preprocessing rather than by the method.

5.3 Serializer

The reduced graph is written as plain text with one process tree per root process, ordered chronologically, with children indented and network and file activity listed beneath each process. Command lines longer than 600 characters are truncated for display only; the gate always checks the full text. The format is identical for every scenario and method.

5.4 Shared Prompt and Parser

One prompt template and one parser serve all three topologies. The model is asked for JSON with a list of malicious events (command and stage) and a narrative. For models that do not emit an internal reasoning trace, grammar-constrained decoding guarantees valid

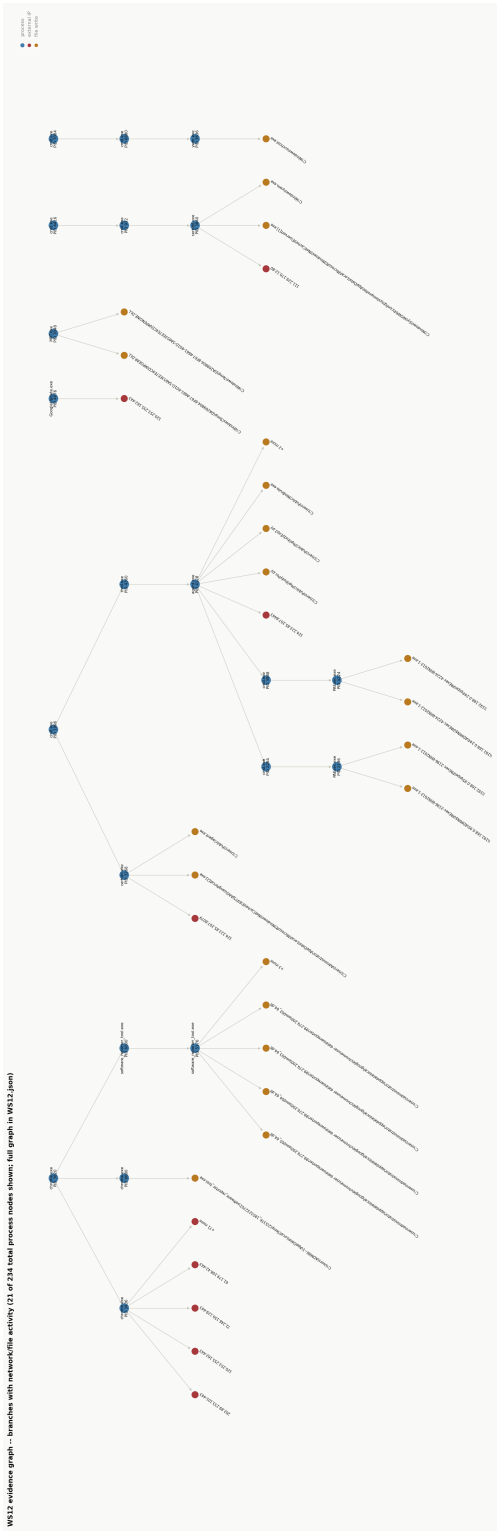


Figure 5.2: Evidence graph of the WS12 scenario after reduction (filtered view). Process nodes, external IP nodes and file-write nodes are drawn in different colours; parent-to-child process edges follow the process tree.

JSON. For the security-tuned model, which writes a reasoning trace first, constrained decoding is disabled because it would forbid the opening token of the trace; the parser removes the trace before extracting the JSON. When a later step fails to parse, the parser keeps the last valid state.

5.5 Method Implementations

CoT, ToT and GoT are implemented in separate modules over the `graph-of-thoughts` framework [5], all using the same prompter and parser. That package imports an attribute that exists only in `openai` before version 1.0 although it never uses it; a stub module avoids downgrading the SDK needed elsewhere. GoT's per-segment steps run as small controllers sharing one model instance so that cost accumulates correctly.

5.6 Improvement Variants

After the baseline we tested four changes, each applied to all three topologies unless stated, and each tested separately:

v2 Output limit raised from 1536 to 4096 tokens, to test whether truncation hid answers.

v3 Short definitions of each stage added to the shared prompt (Table 3.1).

v4 A per-process triage call before labelling: the model first marks which processes look suspicious, and reasoning sees only those. Triage output is cached per scenario, model and seed and reused by all three topologies, so they see identical triaged evidence.

v5 GoT only: the Aggregate call receives the cross-segment structural links (Section 5.6).

5.7 Scorer

The scorer (Section 6.4) was written and unit-tested against hand-verified known-good and known-bad WS12 outputs before any reasoning method existed, and then locked. Two defects were found by the tests and fixed for all methods at once (Chapter 7).

CHAPTER 6

DATA, EVALUATION PROTOCOL AND IMPLEMENTATION

Results are only as trustworthy as the protocol behind them. This chapter specifies the data, the ground truth, the models, the experimental grid, the metrics and the software used for every number in Chapter 8.

6.1 Dataset

6.1.1 Source and Collection

We use the Simulated-Data release that accompanies NODLINK [1]. Each scenario is a monitored host on which the dataset's authors executed a known multi-stage attack, so the log contains the genuine background activity of a working machine (browsers, database services, system processes) mixed with attack activity. For each scenario the release provides the raw audit log (`anomaly.json`, one JSON record per event), annotation files with the attack commands and their times, and an attack-analysis spreadsheet that names the stage of each step. Five scenarios are used (Table 6.1). WS12 is a Windows server and Ubuntu is a Linux host. Sidewinder, FIN6 and APT29 emulate three different threat groups but were executed on the same Windows 10 host, so they share one raw log.

Table 6.1: Scenarios, annotated attack events and the models run in the baseline grid.

Scenario	Platform	GT events	Baseline models
WS12	Windows Server 2012	6	Llama, Qwen
Ubuntu	Linux	8	Llama, Qwen
Sidewinder	Windows 10 (shared log)	15	Llama
FIN6	Windows 10 (shared log)	23	Llama
APT29	Windows 10 (shared log)	37	Llama
Total		**89**	

6.1.2 Raw Log Statistics

Table 6.2 shows the size of each log and the effect of reduction. Across the five scenarios the pipeline streams about 8.7 million raw events (the shared Windows 10 log is counted once per scenario), and the whole graph-building step takes about 24 seconds.

Table 6.2: Raw log size and the effect of reduction. "Process starts" are process-creation events; "collapsed" counts exact repeats merged into one node.

Scenario	Raw events	Process starts	Collapsed	Nodes	Edges
WS12	1,087,311	3,139	2,905	234	255
Ubuntu	3,832,342	2,901	2,515	386	478
Windows 10 (Sidewinder, FIN6, APT29)	1,878,778	2,365	1,741	624	933

6.1.3 Reduced Evidence Graph

After reduction the graph has three edge types: `spawned`, `connected_to` and `wrote` (Section 5.2). For WS12, 31,971 raw external TCP events collapse into 79 `connected_to` edges and 3,335 raw file writes into 24 `wrote` edges, because one long-lived connection writes a log line per packet. Token counts use the `cl100k_base` tokenizer and are larger than our initial plan of about 8K tokens because every process-start node is kept by design.

Table 6.3: Composition of the reduced evidence graph and the length of its serialization.

Scenario	Nodes	Edges	spawned	connected_to	wrote	Tokens
WS12	234	255	152	79	24	16,197
Ubuntu	386	478	328	2	148	19,411
Sidewinder, FIN6, APT29	624	933	453	229	251	58,396

6.1.4 Ground-Truth Statistics

The 89 annotated attack events cover ten stage names (Table 6.4). WS12 and Ubuntu concentrate on Initial Access and Lateral Movement (8 of their 14 combined events, 57%), while the three Windows 10 scenarios are dominated by Execution, Defense Evasion and Discovery. The attack is a small share of its graph: 6 of 234 nodes (2.6%) in WS12 and 37 of 624 (5.9%) for APT29. This sparsity is why detection, and not only labelling, is hard. The class counts are small and uneven, so we report stage metrics aggregated over events and not per-stage F_1.

Table 6.4: Ground-truth events per stage and scenario.

Stage	WS12	Ubuntu	Sidewinder	FIN6	APT29	Total
Initial Access	2	3	0	0	1	6
Execution	0	0	3	6	5	14
Persistence	1	1	2	2	5	11
Privilege Escalation	0	1	0	2	4	7
Defense Evasion	0	0	2	5	8	15
Credential Access	1	1	0	3	1	6
Discovery	1	0	6	3	7	17
Lateral Movement	1	2	0	0	1	4
Collection	0	0	0	2	1	3
Command and Control	0	0	2	0	4	6
Total	6	8	15	23	37	89

Shared-log caveat. The reduced evidence files of Sidewinder, FIN6 and APT29 are byte-identical (624 nodes, 933 edges); only the ground truth differs. For these three the model must choose one of three narratives from the same evidence, which is a harder and different task from WS12 and Ubuntu. We keep the scenarios and treat this as an interpretive limit.

6.1.5 Ground-Truth Construction

Each attack event was assigned a stage and a match substring by reading the scenario's annotation files next to its attack-analysis spreadsheet. We did this by hand because the identifier join differs by scenario. In WS12 the spreadsheet uses a different, non-sequential ID space, so matching by ID silently gives wrong stages. In Ubuntu the IDs align but the rows are out of numeric order. In the other three they align, yet one command text still differs slightly between the two sources. Two nodes cannot be scored because no matching text is present in the log; they are excluded for every method.

6.1.6 Data Pitfalls

- The dataset field `is_warn` encodes the ground-truth label. It is never used as evidence or as a filter.
- Windows and Linux logs have structurally different schemas and need separate parsers.
- PID reuse overwrote graph nodes in the first implementation; nodes are now keyed on process identity.
- Browser subprocess noise inflated the node count roughly tenfold over the planning estimate.

6.2 Models

Table 6.5 lists the models and the constraint that decided how each is used. The models are open-weight, run locally with vLLM, and are not fine-tuned by us.

Table 6.5: Models and the constraints that decided their use.

Model	Role	Constraint
Llama 3.1 8B [11]	Main baseline	131K native context; all five scenarios at a 65,536 cap
Qwen 2.5 7B [12]	Second family	32K native context; WS12 and Ubuntu only
Qwen 2.5 32B (AWQ)	Scale test	Needs one A100 80 GB
Llama 3.1 70B (AWQ)	Scale test	Needs one A100 80 GB
Llama-Primus-Reasoning 8B	Security-tuned, same architecture as Llama 8B	Writes a reasoning trace; constrained decoding off

Two further models were tried and dropped. Phi-4 has a 16K context window, smaller than even the WS12 evidence. Phi-3.5-mini failed to produce parseable output on WS12 across four symmetric fixes (grammar-constrained JSON, an explicit instruction with a worked example, restricted whitespace, and a repetition penalty); we stopped there rather than tune for one model. Forcing Qwen 7B beyond its native context with a position override was rejected because it can silently produce invalid output.

6.3 Experimental Grid

The baseline grid has three methods and three seeds over Llama 3.1 8B on five scenarios and Qwen 2.5 7B on two, giving 63 runs. Variants v2, v3 and v4 repeat this grid with one change each (63 runs each); v5 reruns GoT only (21 runs). The scale test runs Qwen 32B and Llama 70B on the same cells with the v4 setup, and the Primus run covers all five scenarios (45 runs).

6.4 Evaluation Metrics

Let the checkable ground-truth events be $g_1, \ldots, g_n$ and the events a method reports be $e_1, \ldots, e_k$. A reported event *matches* a ground-truth event if the ground-truth match substring occurs in the reported command text, after repeated backslashes are collapsed. A match is *stage-correct* if the stage label is also right.

6.4.1 Event-Level Precision, Recall and F1

$$P = \frac{\#\{\text{reported events that match some } g\}}{k}, \tag{6.1}$$

$$R = \frac{\#\{g \text{ matched by some reported event}\}}{n}, \tag{6.2}$$

$$F_1 = \frac{2PR}{P+R}. \tag{6.3}$$

Precision is counted from the method's side and recall from the answer key's side, so one substring that legitimately matches several commands, or two ground-truth nodes sharing a substring (as with APT29's reused scheduled-task command), is not penalised by a one-to-one matching rule. *Why these metrics:* the first question an analyst asks is whether the true attack steps were found (recall) and how much noise must be read to find them (precision); F_1 summarises both for ranking.

6.4.2 Stage-Level Recall

$$\mathrm{SR} = \frac{\#\{g \text{ matched by a reported event with the correct stage}\}}{n}. \tag{6.4}$$

Because "missed entirely" and "found but mislabelled" are different failures, we also report a partial-credit recall in which a ground-truth event scores 1 if found with the correct stage, 0.5 if found with a wrong stage and 0 if not found. *Why:* stage recall is the quantity the hypothesis is about, and the gap between R and SR isolates labelling errors from detection errors. Exact SR is the headline.

6.4.3 Hallucination Rate

$$H = \frac{\#\{\text{indicators in the output that do not occur in the evidence}\}}{\#\{\text{indicators in the output}\}}, \tag{6.5}$$

where indicators are IPv4 addresses and file names of the form `name.ext` found by regular expression in the narrative and reported commands. *Why:* a fabricated address or file name sends an analyst after something that does not exist, so unsupported claims are scored on their own and not folded into precision. The file-name pattern requires an extension that starts with a letter so that fragments of an IP address are not mistaken for file names.

6.4.4 Why Not One Composite Score

Detection (R), labelling (SR), noise (P) and fabrication (H) fail for different reasons and respond to different interventions (Chapter 8). A composite would hide which of them moved, so we never combine them.

6.4.5 Cost and Averaging

Cost is reported as LLM calls, prompt and completion tokens, and wall-clock seconds. Scores are averaged over seeds within a scenario, model and method, then over cells. Grand means weight each run equally, so scenarios run for two models weigh more; per-scenario tables are therefore also given.

6.5 Statistics

Methods and variants are compared with the paired Wilcoxon signed-rank test described in Section 4.4. With three seeds per cell, small effects cannot be separated from seed variation, and we say so where it applies. No result is called significant without a reported p.

6.6 Software and Hardware

Table 6.6: Software and hardware.

Item	Value
GPUs (development)	2× RTX 6000 Ada, 48 GB (shared node)
GPU (variant and scale jobs)	1× A100 80 GB through SLURM
Inference engine	vLLM 0.26.0
GoT framework	graph-of-thoughts 0.0.2
Environment	Python 3.10, managed with `uv`

The home directory is network storage and breaks Triton and vLLM compilation caches, so those caches are placed on local disk. Because the node is shared, each model's GPU memory share is capped, and SLURM jobs size the share from the memory that is free.

CHAPTER 7

VERIFICATION, AUDIT AND CORRECTIONS

A comparison that differs only in reasoning topology is easy to break by accident. This chapter lists the defects found while building and running the pipeline, because each would have changed the results and because a reader should be able to see that the fixes are real.

7.1 Defects in Data Handling

- **PID reuse.** WS12 has 3,139 process-start events but only 593 distinct PIDs. Keying nodes on PID alone silently overwrote earlier processes, including any that were part of the attack. Nodes are now keyed on process identity including start time.
- **Label leakage.** The dataset field `is_warn` carries the ground-truth label. It is never read.
- **Two raw schemas.** Windows ETW and Linux sysdig/Falco records differ structurally; one parser could not serve both.
- **Edge explosion.** One database process produced 23,010 raw TCP send events for 64 distinct targets. Edges are now deduplicated to one per process and target.
- **Evidence bulk.** Browser helper processes raised WS12's node count about tenfold; exact repeats are now collapsed, and the hard gate confirms that no attack event is lost.
- **Ground-truth text mismatch.** For one WS12 node the match text agreed with the annotation file but not with the raw log. The raw log is the authority, because it is what the model sees.

7.2 Defects in the Scorer

Unit tests with a known-good and a known-bad WS12 case caught a hallucination pattern that matched numeric fragments of an IP address as file names; the test asserted the exact set of missing indicators, which is what exposed it. A later run exposed a backslash mismatch between model output (single backslash in a network path) and ground truth (double), which counted a correct detection as a miss. Both fixes apply to all methods at once and the unit tests were re-run.

7.3 Defects in the Methods

- **GoT segmentation.** The first version split the evidence by root-block *count*. One segment received about 78% of the text and all six WS12 attack events while three others were nearly empty. GoT missed every WS12 event, and checking that large segment's own independent output (which also found none) located the cause. Balancing by character count fixed it, and GoT's WS12 event recall rose from 0 of 6 to 1 of 6 on the test run.
- **Unparseable refinement.** In one Llama run the Refine step failed to parse. The parser now keeps the last valid state rather than discarding it; that preserved the best result seen at that stage.
- **Library incompatibility.** The `graph-of-thoughts` package imports an attribute that exists only in `openai` before version 1.0. A stub avoids a downgrade.

7.4 Defects in Infrastructure

- **Cache on network storage.** vLLM and Triton compilation failed with `errno 61` on the network home directory; caches are now on local disk.
- **Shared GPU.** Default vLLM memory settings would exhaust memory used by other users; the share is now capped per model and sized from free memory.
- **Silent job failure.** The first SLURM job for the security-tuned model reported `COMPLETED` after 76 seconds although the model had failed to load (an incomplete offline cache combined with offline mode). The grid runner now exits non-zero when any model process fails, and the model was downloaded again before rerunning.
- **Context limits.** Phi-4 and Qwen 7B cannot read the longest scenarios, so the grid is deliberately asymmetric (Chapter 6).

7.5 Claims Withdrawn or Qualified

We initially expected GoT's Aggregate to give the best stage recall. The data do not support this (Chapter 8). The three Windows 10 scenarios are not three independent tests, because they share their evidence. And a second command chain that every method reports in WS12 is not hallucination: we checked it against WS12's official attack description, and it is real but unannotated activity (a different command-and-control address and a local account creation). It lowers precision about equally for all methods and is not a topology effect.

CHAPTER 8

RESULTS

All numbers are means over the runs named in each caption. p-values come from the paired Wilcoxon signed-rank test of Section 4.4. Tables marked as generated are produced by `thesis/scripts/make_figures.py` directly from the stored run records.

8.1 Experiments Run

Table 8.1: Inventory of experiments.

Experiment	Setup	Runs	Purpose
Baseline (v1)	Llama 8B (5 scenarios), Qwen 7B (2)	63	RQ1
v2	Output limit 4096 tokens	63	Truncation check
v3	Stage definitions in prompt	63	Stage vocabulary
v4	Per-process triage added	63	False-positive control
v5	GoT with cross-segment links	21	Aggregate mechanism
Scale (v4 setup)	Qwen 32B, Llama 70B (AWQ)	63	Model size
Scale, no triage	Llama 70B	63	Input-length effect
Primus (v4 setup)	Llama-Primus-Reasoning 8B	45	RQ3

8.2 Baseline: Topology Comparison (RQ1)

Table 8.2 and Figure 8.1 give the baseline over all 63 runs (21 per method).

Table 8.2: Baseline results by method (mean of 21 runs each).

Method	Precision	Recall	Event F_1	Stage recall	Partial SR	Halluc.
CoT	0.179	0.200	0.181	0.049	0.096	0.095
ToT	0.235	0.263	0.234	0.018	0.115	0.190
GoT	0.223	0.375	0.267	0.061	0.192	0.157

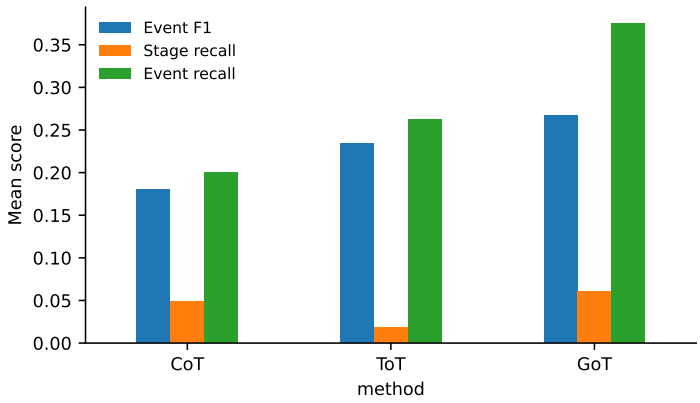


Figure 8.1: Baseline mean event F_1, stage recall and event recall by method.

GoT has the highest mean event recall and F_1, and ToT the highest precision. For stage recall no pairwise difference is significant, while GoT's event-recall advantage is significant before multiple-test correction (Section 8.3). Table 8.3 splits the same runs by model; Qwen 2.5 7B has an exact stage recall of exactly 0.000 under CoT and ToT.

Table 8.3: Baseline results by model and method.

Model	Method	Precision	Recall	Event F_1	Stage recall
Llama 3.1 8B	CoT	0.152	0.177	0.154	0.069
	ToT	0.217	0.258	0.218	0.025
	GoT	0.188	0.324	0.222	0.053
Qwen 2.5 7B	CoT	0.245	0.256	0.246	0.000
	ToT	0.280	0.274	0.274	0.000
	GoT	0.312	0.505	0.380	0.079

By scenario (Figure 8.2), mean event F_1 over all baseline runs is 0.362 for WS12, 0.280 for Ubuntu, 0.116 for Sidewinder, 0.107 for FIN6 and 0.084 for APT29. The three scenarios that share one Windows 10 log score far lower than the two that do not.

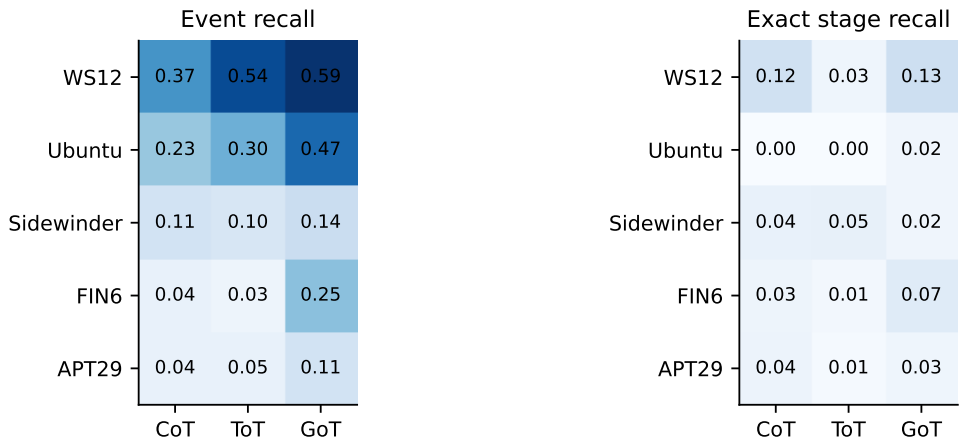


Figure 8.2: Baseline event recall and exact stage recall by scenario and method.

8.3 Tests of the Hypothesis

Table 8.4 applies the paired test of Section 4.4 to every experiment. Δ is the mean difference (first method minus second) over the n matched cells.

Table 8.4: Paired Wilcoxon tests between topologies (generated). Δ is the mean difference; n is the number of matched (scenario, model, seed) cells.

		GoT vs CoT		GoT vs ToT		ToT vs CoT		
Experiment	Metric	Δ	p	Δ	p	Δ	p	n
Baseline (v1)	Stage recall	+0.011	0.386	+0.043	0.113	-0.031	0.161	21
	Event recall	+0.175	0.009	+0.113	0.023	+0.063	0.278	21
v3 stage definitions	Stage recall	+0.089	0.049	+0.071	0.256	+0.017	0.445	21
	Event recall	+0.166	0.013	+0.077	0.232	+0.090	0.031	21
v4 triage	Stage recall	+0.022	0.727	+0.042	0.102	-0.020	0.328	21
	Event recall	+0.048	0.478	+0.069	0.191	-0.022	0.629	21
Scale: Qwen 32B, Llama 70B	Stage recall	+0.049	0.201	+0.085	0.028	-0.036	0.260	21
	Event recall	+0.142	0.007	+0.149	0.006	-0.007	0.975	21
Llama-Primus 8B	Stage recall	+0.030	0.348	+0.019	0.600	+0.011	0.684	15
	Event recall	+0.072	0.126	-0.003	0.859	+0.075	0.123	15

Of the 30 comparisons in the table, 8 have an uncorrected $p < 0.05$, and they are not spread evenly across the two metrics.

- **Stage recall, the metric named in the hypothesis.** Only two of the 15 comparisons are below 0.05: GoT against CoT in v3 ($\Delta = +0.089$, $p = 0.049$) and GoT against ToT in the scale experiment ($\Delta = +0.085$, $p = 0.028$). In the baseline, v4 and Primus experiments GoT's advantage in stage recall is small and not significant (p from 0.10 to 0.73).

- **Event recall.** GoT is significantly higher than CoT in the baseline ($\Delta = +0.176$, $p = 0.009$), v3 ($+0.167$, $p = 0.013$) and scale ($+0.142$, $p = 0.007$) experiments, and higher than ToT in the baseline ($+0.113$, $p = 0.023$) and scale ($+0.149$, $p = 0.006$) experiments. It is not significant under v4 or for Primus.

These are 30 tests, so some small p-values are expected by chance. After Bonferroni correction (threshold $0.05/30 = 0.0017$) none survives, and the smallest Holm-adjusted p is 0.18. We also pair each seed of a scenario as a separate observation, which treats repeated runs on the same scenario as independent and makes the tests somewhat anti-conservative. The honest reading is therefore: the primary prediction (higher stage recall for GoT) is not confirmed, and H0 is not rejected; GoT does show a consistent *direction* of higher event recall, which is not established as a real effect at this sample size.

8.4 Comparison across Models and Topologies

Figure 8.3 and Table 8.5 compare the five models under the same v4 setup for each topology. Qwen 7B and Llama 8B cover the cells described in Chapter 6; Qwen 32B and Llama 70B and Primus use the cells their context allows.

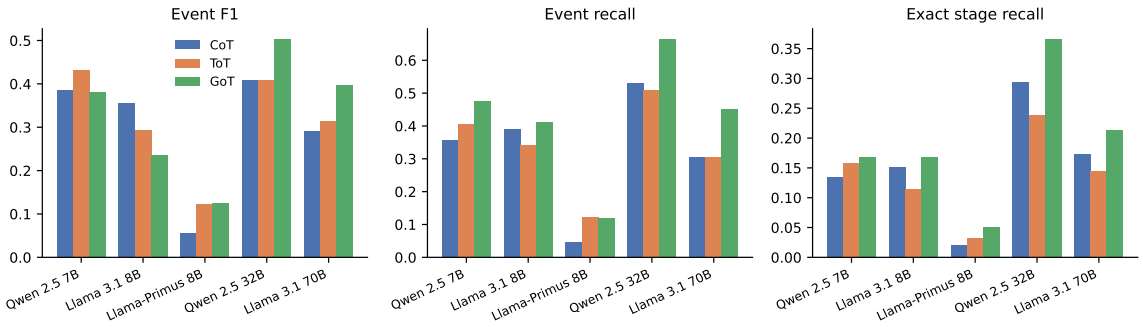


Figure 8.3: Event F_1, event recall and exact stage recall for each model and topology under the v4 setup.

Table 8.5: Models and topologies under the v4 setup (generated).

Model	Method	Runs	Precision	Recall	Event F_1	Stage recall
Qwen 2.5 7B	CoT	6	0.493	0.357	0.385	0.135
	ToT	6	0.538	0.405	0.432	0.159
	GoT	6	0.340	0.475	0.380	0.169
Llama 3.1 8B	CoT	15	0.343	0.390	0.355	0.151
	ToT	15	0.289	0.341	0.294	0.114
	GoT	15	0.203	0.410	0.236	0.169
Llama-Primus 8B	CoT	15	0.097	0.048	0.055	0.021
	ToT	15	0.134	0.123	0.124	0.031
	GoT	15	0.190	0.119	0.124	0.050
Qwen 2.5 32B	CoT	6	0.356	0.532	0.408	0.294
	ToT	6	0.368	0.508	0.410	0.238
	GoT	6	0.412	0.663	0.502	0.365
Llama 3.1 70B	CoT	15	0.373	0.305	0.291	0.173
	ToT	15	0.396	0.305	0.314	0.145
	GoT	15	0.370	0.452	0.396	0.214

Two patterns stand out. First, the gaps between models are larger than the gaps between topologies within a model. Second, GoT has the highest event F_1 and stage recall for both large models (Qwen 32B: 0.502 and 0.365; Llama 70B: 0.396 and 0.214). GoT gives the best event recall for both large models and for Llama 8B, but its precision is lower than CoT's for the 7B and 8B models, so it finds more events at the price of more false ones.

8.5 Improvement Rounds

Figure 8.4 shows the pooled effect of each change and Table 8.6 gives the paired tests between successive variants.

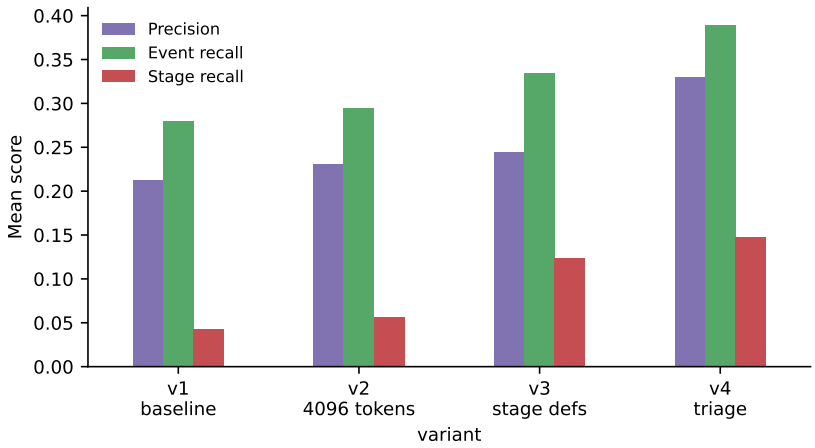


Figure 8.4: Pooled precision, event recall and stage recall by variant (63 runs each).

Table 8.6: Paired tests between successive variants (generated).

Step	Metric	Before	After	p	n
v1 to v2	Stage recall	0.043	0.056	0.1011	63
	Precision	0.212	0.231	0.5467	63
	Event recall	0.279	0.295	0.5439	63
	Event F1	0.227	0.239	0.6114	63
v2 to v3	Stage recall	0.056	0.123	0.0015	63
	Precision	0.231	0.245	0.4232	63
	Event recall	0.295	0.334	0.0515	63
	Event F1	0.239	0.268	0.1038	63
v3 to v4	Stage recall	0.123	0.147	0.3445	63
	Precision	0.245	0.329	0.0400	63
	Event recall	0.334	0.390	0.0762	63
	Event F1	0.268	0.325	0.0802	63

- **v2.** Raising the output limit changes nothing significantly, so truncation was not the bottleneck.
- **v3.** Stage definitions double exact stage recall, from 0.056 to 0.123 ($p = 0.0015$). This is the first significant result, and it addresses the labelling failure diagnosed in Chapter 9.
- **v4.** Triage raises precision from 0.245 to 0.329 ($p = 0.04$) and event F_1 from 0.268 to 0.325 ($p = 0.08$). Detection barely moves (19.9% to 21.3% of true events found), so the gain comes mainly from fewer false positives.
- **v5.** Giving GoT's Aggregate the cross-segment links raises GoT's event F_1 from 0.277 to 0.350 ($p = 0.34$) and Llama's event recall from 0.41 to 0.54 ($p = 0.055$). Neither is significant, and GoT still does not separate from CoT and ToT.

8.6 Scaling the Model

- Qwen 7B to 32B: exact stage recall 0.15 to 0.30 ($p = 0.012$) and event recall 0.41 to 0.57 ($p = 0.017$).
- Llama 8B to 70B: precision 0.28 to 0.38 ($p = 0.022$) and hallucination 0.19 to 0.06 ($p = 0.001$), with recall flat. The 70B's stricter triage dropped 22% of true attack events before reasoning began.
- Llama 70B without triage is worse than with it: event F_1 falls from 0.33 to 0.23 ($p = 0.0015$). Figure 8.5 shows why: on the longest inputs the 70B's recall collapses.

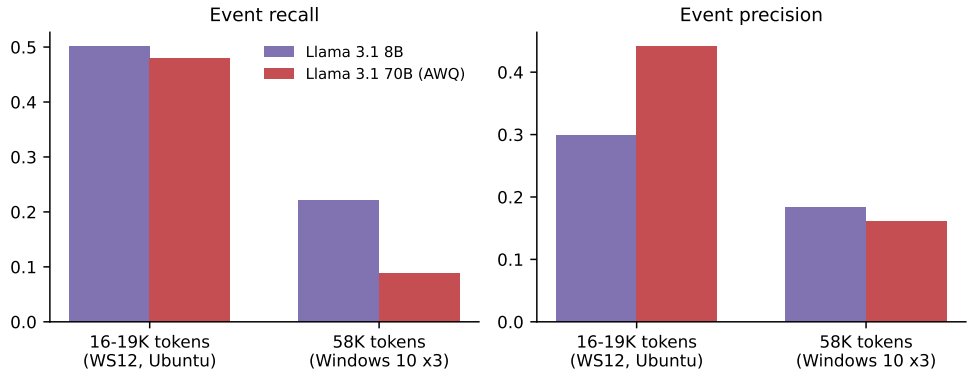


Figure 8.5: Llama 70B against Llama 8B without triage, split by input length. At 58K tokens the larger model's recall falls below the smaller one's.

8.7 Security-Tuned Model (RQ3)

Llama-Primus-Reasoning has the size and architecture of Llama 3.1 8B but was further trained on cyber-security text and writes a reasoning trace before its answer. It was run under the v4 setup with the Llama 3.1 8B triage reused, so it and the comparison model read identical triaged evidence.

Table 8.7: Llama-Primus-Reasoning by method, v4 setup (generated).

Method	Precision	Recall	Event F_1	Stage recall	Halluc.
CoT	0.097	0.048	0.055	0.021	0.065
ToT	0.134	0.123	0.124	0.031	0.031
GoT	0.190	0.119	0.124	0.050	0.165
All (45 runs)	0.140	0.096	0.101	0.034	0.087

Over all 45 runs the security-tuned model is worse than Llama 3.1 8B on the same evidence (Table 8.8). Paired over the 45 matched cells, event F_1 falls from 0.295 to 0.101,

event recall from 0.381 to 0.096 and exact stage recall from 0.145 to 0.034; all are significant (recall and F_1 $p < 0.001$, stage recall $p = 0.0001$, precision $p = 0.002$). Its hallucination rate is lower (0.087 against 0.194, $p = 0.005$), but only because it reports fewer events.

Table 8.8: Llama-Primus-Reasoning against Llama 3.1 8B, v4 setup, 45 matched runs each.

Model	Precision	Recall	Event F_1	Stage recall	Halluc.
Llama 3.1 8B	0.278	0.381	0.295	0.145	0.194
Llama-Primus-Reasoning	0.140	0.096	0.101	0.034	0.087

Output-format failures. 17 of the 45 runs (38%) produced no parseable answer (Figure 8.6): 10 of 15 CoT, 11 of 15 ToT and 7 of 15 GoT runs parsed. All Sidewinder runs scored zero. The model writes a long reasoning trace first (about 2,000 completion tokens for CoT and 10,000 for GoT), and because constrained decoding is off for it, nothing forces a valid answer. On the 28 runs that parsed, event F_1 is 0.126, event recall 0.120 and stage recall 0.031, so format failures explain part of the deficit but not all of it.

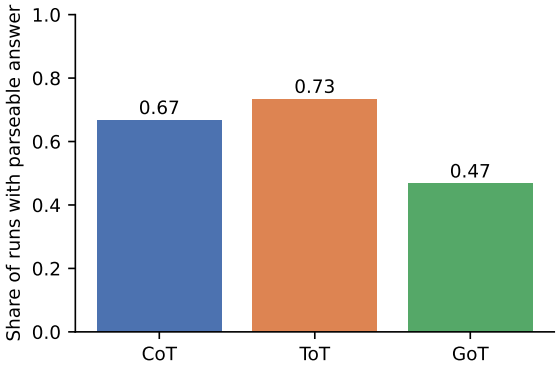


Figure 8.6: Share of Llama-Primus-Reasoning runs that produced a parseable answer, by method (15 runs each).

Topology. Within Primus, GoT and ToT have higher recall than CoT (0.119 and 0.123 against 0.048), but no comparison is significant (Table 8.4).

8.8 Accuracy and Cost

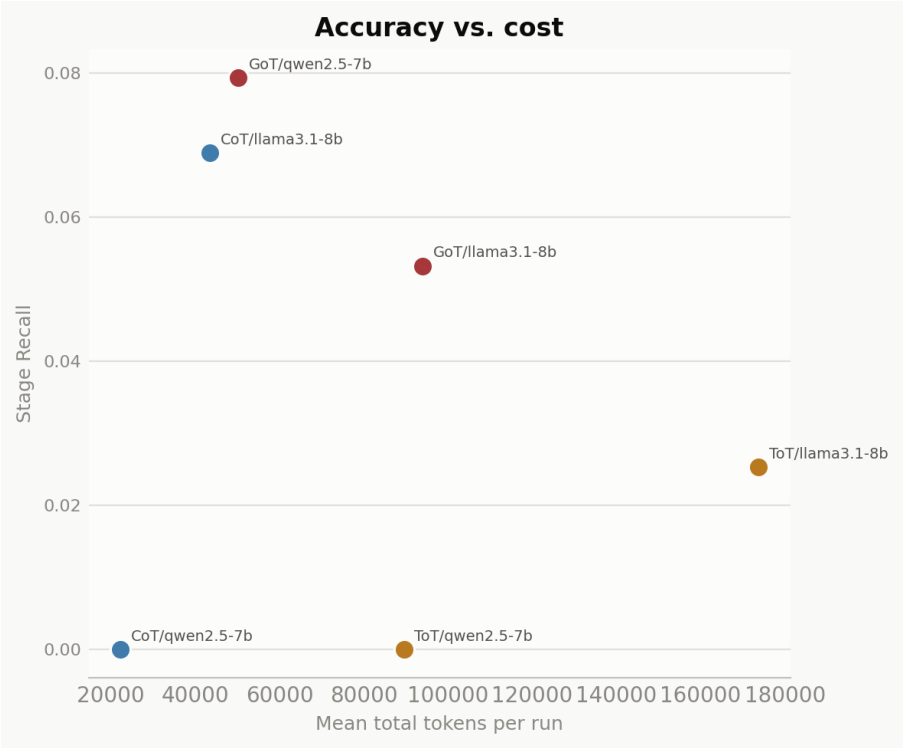


Figure 8.7: Stage recall against cost for each method and model, from the baseline grid.

Table 4.1 gives the baseline cost. GoT costs about twice the tokens of CoT and four times its wall-clock time, and ToT costs about four times the tokens of CoT. Neither extra cost buys a statistically significant accuracy gain in our data.

8.9 Summary of Results

1. On the primary metric, stage recall, GoT is not reliably better than CoT or ToT: 2 of 15 comparisons are below 0.05 uncorrected, and none survives correction.
2. GoT has consistently higher event recall (uncorrected $p < 0.05$ in 5 of 10 comparisons against CoT and ToT), at the cost of lower precision at 7B–8B; none survives Bonferroni correction.
3. Stage definitions in the prompt (stage recall doubled), triage (precision $+0.08$) and model size (Qwen stage recall doubled) have larger and clearer effects than topology.
4. A security-tuned 8B model did worse than the general one, partly because of output-format failures.
5. Input length materially affects accuracy and is confounded with GoT's segmentation.

CHAPTER 9

DISCUSSION: WHERE AND WHY RECONSTRUCTION FAILS

9.1 Answers to the Research Questions

RQ1: does GoT beat CoT and ToT on stage recall? Not reliably. GoT's mean stage recall is highest in most settings, but only 2 of 15 paired comparisons are below $p = 0.05$ before correction and none survives it (Table 8.4), so H0 is not rejected for the primary metric. GoT does show a more consistent advantage in event recall (uncorrected $p < 0.05$ in 5 of 10 comparisons), paid for with lower precision at 7B–8B. Three seeds per cell and the pairing of seeds as observations limit what can be concluded.

RQ2: where does reconstruction fail, and what changes the numbers? Four failure modes explain most of the loss, described below. The interventions that worked address them directly: stage definitions address labelling, triage addresses noise, and a larger model addresses detection and labelling together.

RQ3: does a security-tuned model change the conclusion? No. Llama-Primus-Reasoning was significantly worse than Llama 3.1 8B on identical evidence, partly through output-format failures, and no topology comparison was significant for it.

9.2 Failure 1: Most Attack Events Are Never Detected

Reading outputs against ground truth, only 15.3% of the 89 true events are ever reported, regardless of topology. The attack is a small fraction of each graph (2.6% of nodes in WS12, 5.9% in APT29), and many of its steps use ordinary administrative utilities that look benign without attacker context. A method cannot aggregate evidence that no step flagged. This also bounds every recall figure in Chapter 8.

9.3 Failure 2: Stage Labels Are Miscalibrated

Among the events that are found, exact stage accuracy is about 20%, against 10% for a random choice among ten stage names. Each model has a stable preference over stage names that does not match this data. Qwen 2.5 7B assigns Initial Access to only 3.9% of the events it reports and Lateral Movement to 1.1%, whereas those two stages are 8 of the

14 (57%) ground-truth events of WS12 and Ubuntu together. This explains Qwen's exact stage recall of 0.000 under CoT and ToT. Figures 9.1 and 9.2 show where predicted and true stages disagree.

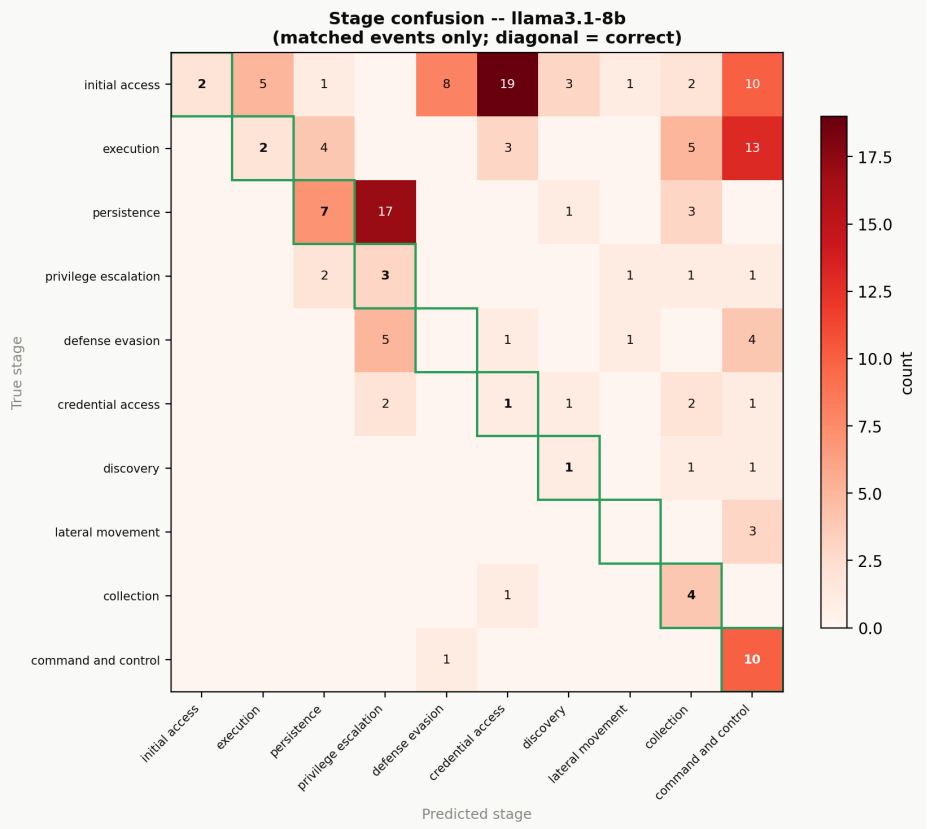


Figure 9.1: Stage confusion matrix for Llama 3.1 8B: true stage against predicted stage over all found events.

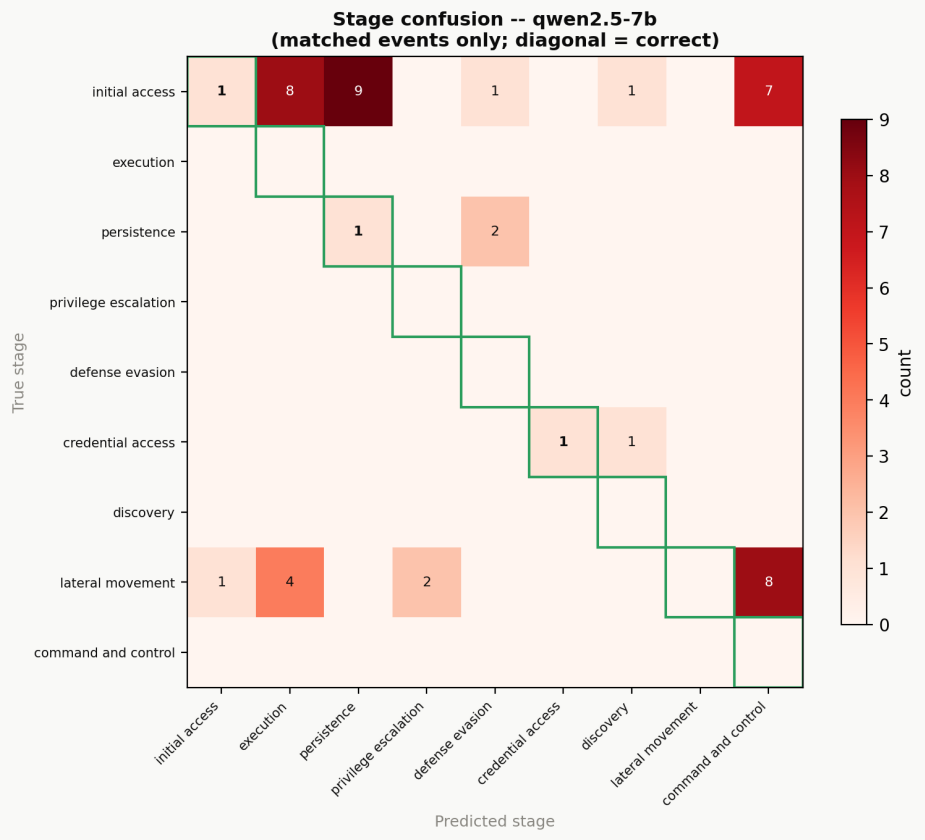


Figure 9.2: Stage confusion matrix for Qwen 2.5 7B.

The fix that worked was to supply the vocabulary: adding stage definitions to the prompt doubled exact stage recall (Section 8.5).

9.4 Failure 3: Evidence the Method Cannot Use or Cannot Tell Apart

9.4.1 The Shared Windows 10 Log

Sidewinder, FIN6 and APT29 share one evidence file, so the model sees the same 624 nodes whichever attack it is asked about. All three score far below WS12 and Ubuntu (Figure 8.2) and Primus scores zero on Sidewinder. This is a limit of the data, not of any topology.

9.4.2 Why the Aggregate Mechanism Did Not Show

The argument for H1 needs two things: attack events must be spread across segments, and each segment's analysis must retain its part. Neither is well supported here.

1. **Low per-segment detection.** With only 15.3% of true events detected at all, the per-segment results rarely contain the pieces to be merged.

2. **Little scattered structure in the data.** A pre-check on WS12 found that all attack events fall into one segment after balanced splitting. Aggregate then has nothing to link, and GoT reduces to analysing one segment and restating it.

3. **Aggregation loses information.** The merge call sees only the per-segment answers, not the evidence behind them, so an event omitted from a segment answer cannot be recovered at that step.

The v5 variant, which gives Aggregate the cross-segment links explicitly, raised GoT's scores but not significantly. That fits a mechanism that works in principle but is starved of inputs.

9.5 Failure 4: Input Length and Output Format

9.5.1 Input Length

The Llama 70B comparison shows that the same model can be strong or weak depending on input length alone (Figure 8.5). On the 16–19K-token inputs it beats the 8B (precision 0.44 against 0.30); on the 58K-token inputs its recall collapses to 0.09 (8B: 0.22). Triage shortens the input and helps; so, plausibly, does GoT's segmentation, because most of its calls read a quarter of the evidence. GoT's consistent advantage in event recall, with lower precision, is what shorter, independent per-segment reads would produce, so it could come from splitting rather than aggregating. A segmented-CoT control, with the same split and a plain concatenation of the segment answers instead of an LLM merge, would separate the two. We did not run it.

9.5.2 Output Format

For the security-tuned model, 38% of runs ended without a parseable answer (Figure 8.6). With constrained decoding off, a long reasoning trace can consume the output budget before any JSON is written. This is a pipeline limitation as much as a model one, which is why we do not read the Primus result as evidence against security tuning in general.

9.6 Failure 5: Unannotated Activity Counts as an Error

All methods independently report a second command chain in WS12 that is not in the annotation. We checked it against the official attack description and it is real activity, so some false positives are genuinely correct findings. They lower precision about equally for all methods, so they do not bias the topology comparison, but they make absolute precision values conservative.

9.7 Status of the Claims

Table 9.1: What the evidence supports.

Claim	Status	Evidence
GoT beats CoT and ToT on stage recall	Not supported	2 of 15 uncorrected $p < 0.05$; none after correction
GoT gives higher event recall	Suggestive	5 of 10 uncorrected $p < 0.05$; none after correction; lower precision
Stage definitions raise stage recall	Supported	0.056 to 0.123, $p = 0.0015$
Triage raises precision	Supported	0.245 to 0.329, $p = 0.04$
Larger model raises stage recall (Qwen)	Supported	0.15 to 0.30, $p = 0.012$
Long input hurts a large model	Supported (one comparison)	70B recall 0.09 against 8B 0.22 at 58K tokens
Security tuning helps	Not supported	Primus F_1 0.101 against 0.295, $p < 0.001$
GoT's gain is due to aggregation	Untested	Needs ablation and segmented-CoT control

9.8 Threats to Validity

- **Statistical power and pairing.** Three seeds per cell cannot detect small effects, and treating each seed of a scenario as a separate paired observation makes the tests anti-conservative. We report 30 topology tests without correction in the table and state which survive correction (none). A non-significant result means "not shown", not "shown to be absent".
- **Few scenarios, one shared log.** Five scenarios, three of which share their evidence, are not five independent samples.
- **Asymmetric grid.** Qwen 7B could run only the two short scenarios, so cross-model comparisons use WS12 and Ubuntu only.
- **Substring matching.** A correct event described with different text would be counted as a miss, for every method equally.
- **Model range.** Only models from 7B to 70B were tested, and one security-tuned model.

9.9 What the Audit Taught Us

The defects of Chapter 7 each had the potential to change the answer silently: a label-leaking field, overwritten nodes, an unbalanced split that gave one GoT segment most of the evidence, and a job that reported success after a crash. In a comparison of this kind, the engineering around the methods deserves as much scrutiny as the methods.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Conclusion

We built a pipeline in which Chain-, Tree- and Graph-of-Thoughts receive identical evidence and are scored identically, and used it to test whether Graph-of-Thoughts improves attack kill-chain reconstruction on provenance logs. Across five scenarios, 89 annotated events and models from 7B to 70B parameters, GoT's stage recall, the metric the hypothesis names, is not reliably higher than that of CoT or ToT (2 of 15 comparisons below 0.05 uncorrected, none after correction), so the hypothesis is not supported and the null hypothesis is not rejected. GoT does recover more events (uncorrected $p < 0.05$ in 5 of 10 comparisons) at lower precision for the 7B and 8B models, which suggests that splitting the evidence, not aggregating it, is what helps. What did matter was the prompt's stage vocabulary (exact stage recall doubled), per-process triage (precision rose from 0.245 to 0.329) and model size (Qwen's stage recall doubled from 7B to 32B). The limiting factors are low detection recall (15.3% of true events found), stage miscalibration (about 20% exact accuracy among found events), input length, and, for the security-tuned model, output-format failures. A security-tuned 8B model did worse than the general model (event F_1 0.101 against 0.295).

10.2 Objectives Revisited

Table 10.1: Status of the objectives of Chapter 1.

No.	Objective	Status
1	Fixed evidence pipeline with hard gate	Done; all five scenarios pass
2	CoT, ToT, GoT on the same models	Done; parseable output on every cell run
3	Event, stage, hallucination and cost metrics	Done; scorer locked before any method
4	Test H1 with paired tests	Done; not supported. The aggregation ablation was not run
5	Accuracy–cost trade-off	Done (Table 4.1, Figure 8.7)
6	Diagnose failures and test improvements	Done (Chapters 8 and 9)

10.3 Future Work

1. **Aggregation ablation and segmented-CoT control.** GoT without Aggregate, and the same split with answers concatenated rather than merged by a model, to separate the effect of splitting from the effect of aggregating.
2. **Data where Aggregate has work to do.** Scenarios in which attack events are spread over segments, reporting how many true event pairs cross a segment boundary.
3. **Calibrating stage labels.** Few-shot examples, or correcting the model's stage preferences on held-out scenarios.
4. **Enforced output format for reasoning models.** Free reasoning followed by a constrained answer-extraction call, so that format failures do not mask a model's detection ability.
5. **More seeds where effects are marginal,** with effect sizes and confidence intervals.
6. **Independent scenarios.** Give Sidewinder, FIN6 and APT29 separate evidence so they become independent tests.
7. **Fine-tuning the best reasoning model and inspecting its layers.** Take the strongest reasoning model from this study and fine-tune it on kill-chain reconstruction, then examine its internal layers (for example which layers and attention heads respond to attack-related processes and stage names) to see where detection and stage labelling succeed or fail and to guide what to train.
8. **Larger and reasoning-specialised models** with a context window that covers the long scenarios.

10.4 Practical Takeaways

For a practitioner with a local model and a long log, our results suggest putting effort first into shortening and filtering the input, supplying the stage vocabulary in the prompt, and using the largest model that fits, before adding multi-call reasoning structures whose benefit we could not demonstrate.

REFERENCES

[1] S. Li, F. Dong, X. Xiao, H. Wang, F. Shao, J. Chen, Y. Guo, X. Chen, and D. Li. NODLINK: An online system for fine-grained APT attack detection and investigation. In *Network and Distributed System Security Symposium (NDSS)*, 2024.

[2] CrowdStrike. 2024 global threat report. Technical report, CrowdStrike, 2024.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[5] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024.

[6] A. Aly, E. Mansour, and A. Youssef. OCR-APT: Reconstructing APT stories from audit logs using subgraph anomaly detection and LLMs. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.

[7] X. Zhang, C. Du, T. Pang, Q. Liu, W. Gao, and M. Lin. Chain of preference optimization: Improving chain-of-thought reasoning in LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[8] Y. Wu, M. Velazco, A. Zhao, M. R. Meléndez Luján, S. Movva, Y. K. Roy, Q. Nguyen, R. Rodriguez, Q. Wu, M. Albada, J. Kiseleva, and A. Mudgerikar. ExCyTIn-Bench: Evaluating LLM agents on cyber threat investigation. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2026.

[9] The MITRE Corporation. MITRE ATT&CK knowledge base. `https://attack.mitre.org`.

[10] F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6): 80–83, 1945.

[11] A. Dubey et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[12] A. Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

CHAPTER A

CONFIGURATION

Table A.1: Per-model settings used in the grids.

Model	Max model length	GPU memory share
Llama 3.1 8B	65,536	0.55 (shared node)
Qwen 2.5 7B	32,768 (native)	capped per model
Llama-Primus-Reasoning	65,536	0.55 default; 0.85 cap on SLURM
Large models (AWQ)	per model	sized from free memory, cap 0.85

Common settings: maximum output 4096 tokens for variants v2 onward (1536 in the baseline); three seeds per cell; one shared prompt and parser; grammar-constrained JSON for models without an internal reasoning trace.

CHAPTER B

REPRODUCING THE RESULTS

B.1 Environment

```
uv venv --python 3.10 && uv pip install -r requirements.txt
export HF_HUB_OFFLINE=1   # models must be pre-downloaded
```

B.2 Pipeline

```
python src/parse_ground_truth.py
python src/build_graph.py && python src/serialize.py
python tests/test_scorer.py
```

B.3 Grids

```
python -u src/run_grid.py --variant v3_stagedefs \
    --prompt-variant stagedefs --max-tokens 4096
python -u src/run_grid.py --variant primus_v4 \
    --models llama-primus-reasoning --prompt-variant stagedefs \
    --max-tokens 4096 --triage --triage-model llama3.1-8b
sbatch slurm/run_primus.sbatch
```

Runs are resumable: a resubmitted job skips runs already in the variant's results file. The runner exits non-zero if any model process fails.

B.4 Figures

```
python thesis/scripts/make_figures.py
```

CHAPTER C

ADDITIONAL OUTPUTS

C.1 Per-Scenario Baseline

Table C.1: Baseline means by scenario (all methods and models pooled).

Scenario	Runs	Event F_1	Event recall	Stage recall
WS12	18	0.362	0.499	0.091
Ubuntu	18	0.280	0.333	0.008
Sidewinder	9	0.116	0.115	0.039
FIN6	9	0.107	0.109	0.039
APT29	9	0.084	0.066	0.024